上讲主要内容:

- HASH
- HASH算法
- HASH安全性
- MAC

《密码学基础》

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN

第12讲 数字签名

消息不可否认性



授课教师：徐国爱

Contents 内容



1 数字签名概述

2 基于RSA的数字签名

3 基于离散对数的数字签名

4 盲数字签名

1.1 数字签名的引言

- 手写签名是一种传统的确认方式，如写信、签订协议、支付确认、批复文件等。在数字系统中同样有签名应用的需求，如假定A发送一个认证的信息给B，如果没有签名确认的措施，B可能伪造一个不同的消息，但声称是从A收到的；或者为了某种目的，A也可能否认发送过该消息。很显然，数字系统的特点决定了不可能沿用原先的手写签名方法来实现防伪造或抵赖，这就是提出了如何实现数字签名的问题。
- 数字签名是电子信息技术发展的产物，是针对电子信息的一种签名确认方法，所要达到的目的是：对数字对象的合法化、真实性进行标记，并提供签名者的承诺。随着信息技术的广泛使用，特别是电子商务、电子政务等快速发展，数字签名的应用需求越来越大。

1.1 数字签名的简介

- 数字签名体制是以电子签名形式存储消息的方法，所签名的消息能够在通信网络中传输。
- 在当今数字化的信息世界里，数字化信息的认证性（来源真实性！）、完整性和不可否认性是实现信息化的基本要求，也决定信息化的普及和推广。数字签名是满足上述要求的主要手段之一，也是现代密码学的主要研究内容之一。
- 数字签名是日常生活中手写签名的电子对应物。

1.1 数字签名与手写签名的不同

- 签名：手写签名是被签文件的物理组成部分；数字签名是连接到被签消息上的数字串。
- 传输方式：数字签名和所签名的消息能够在通信网络中传输；手写签名使用传统的安全方式传输。
- 验证：手写签名是通过将它与真实的签名进行比较来验证；而数字签名是利用已经公开的验证算法（含公开参数！）来验证。
- 数字签名的复制是有效的；而手写签名的复制品是无效的。
- 手书签字是模拟的，且因人而异；数字签字是0和1的数字串，因消息而异。（也因人而异、通常还要因时间而不同！）

1.2 数字签名的概念

- 所谓数字签名(Digital Signature)，也称电子签名，是指附加在某一电子信息中的一组特定的符号或代码，它是利用数学方法和密码算法对该电子信息进行关键信息提取（HASH）并进行加密而形成的，用于标识签发者的身份以及签发者对电子信息的认可，并能被接收者用来验证该电子信息在传输过程中是否被篡改或伪造。

1.2 数字签名的理解

- 数字签名由公钥密码发展而来，作为密码学基本元语，它在信息安全，主要包括身份认证、数据完整性、不可否认性等方面有着重要应用。
- 数字签名的目的是提供一种手段，使得一个实体把他的身份与某个信息捆绑在一起。一个消息的数字签名实际上是一个数，它仅仅依赖于签名者知道的某个秘密(或称为私钥)，也依赖于被签名信息的本身。
- 数字签名基于两条基本的假设：一是私钥是安全的，只有其拥有者才能获得；二是产生数字签名的惟一途径是使用私钥。

1.2 数字签名的相关对比

- 与消息加密不同：消息加密和解密可能是一次性的（时效性！），它要求在解密之前是安全的；而一个签字的消息可能作为一个法律上的文件，如合同等，很可能在对消息签署多年之后才验证其签字，且可能需要多次验证此签字。
- 与消息认证不同：数字签名也是一种消息认证技术，它基于非对称密钥，消息认证码基于对称密钥，且消息认证码的处理速度比数字签名快得多。但是，消息认证码无法实现不可否认性。

1.3 数字签名的安全要求

- 签名是可以被验证的

  接受者能够核实签名者对消息的签名。

- 签名是不可伪造的

  除了签名者，任何人(包括接受者)不能伪造消息的签名。

- 签名是不可重用的

  同一消息不同时刻其签名是有区别的。

- 签名是不可抵赖的

  签名者事后不能抵赖对消息的签名，出现争议时，第三方可解决争端。

1.3 数字签名算法的实现要求

- 签名的产生必须使用签名者独有的一些信息(私钥)以防伪造和否认，同时，要求保证独有的一些信息的安全性。
- 签名的产生应较为容易。
- 签名的识别和验证应较为容易。
- 对已知的数字签名构造一新消息的数字签名在计算上是不可行的。(Hash的安全)
- 对已知消息构造一假冒的数字签名在计算上都是不可行的。(私钥的安全)

1.4 数字签名方案的组成

- 数字签名方案是由五元组组成的，
- 即{P,S,K,Sign,Ver}。
- P:明文空间；
- S:签名空间；
- K:密钥空间；
- Sign:签名算法；
- Ver:验证算法；

1.4 数字签名的组成

- 系统初始化

  生成数字签名方案用到的所有参数（公私钥对）。

- 签名生成算法

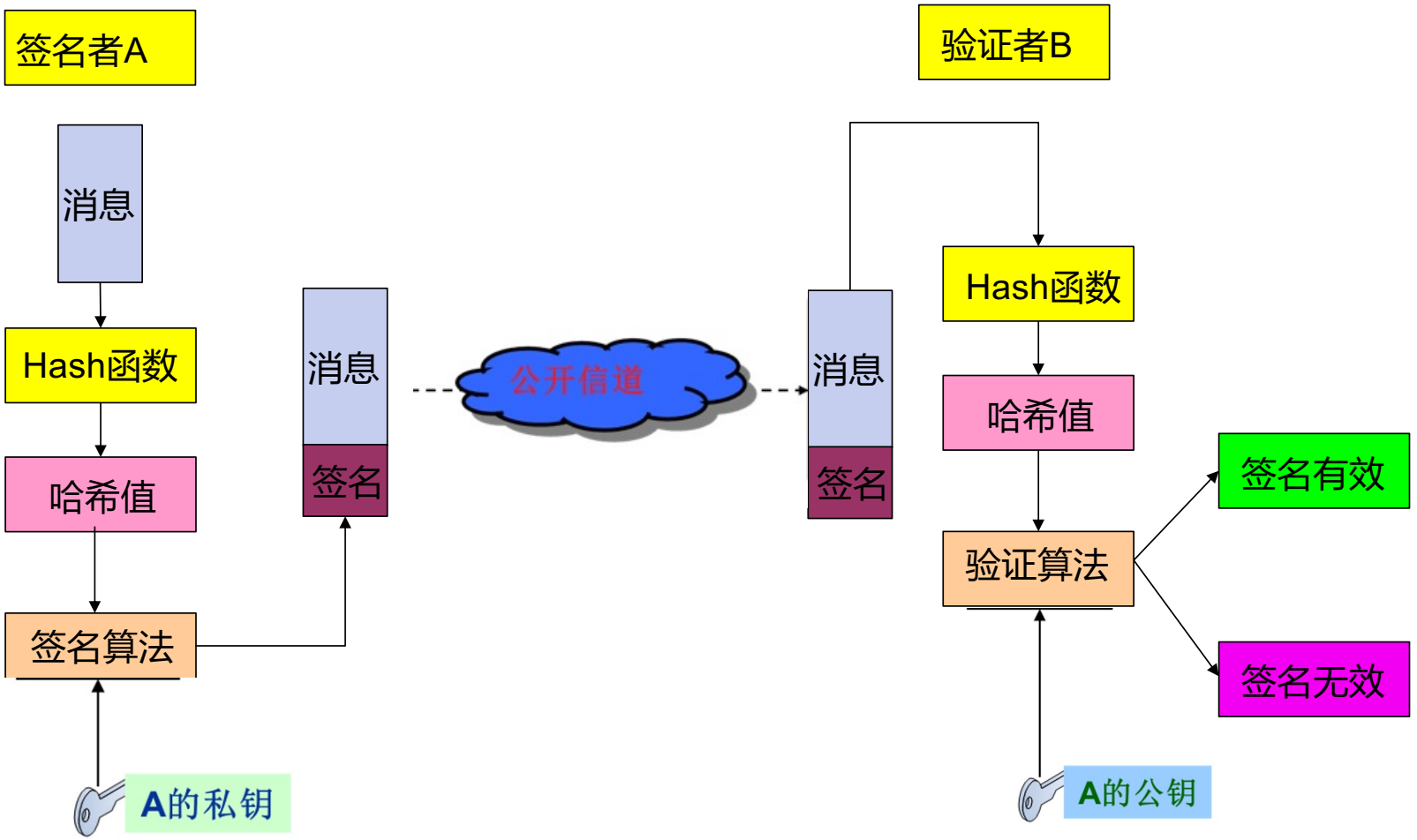
  用户利用给定的算法对消息产生签名**s=Sign私钥(m)**。

- 签名验证算法

  验证者利用公开的验证方法对给定消息的签名进行验证，得出签名的有效性。**Ver公钥(s,m)=0或1**。

1.4 数字签名的实现过程

哈爾濱工業大學(深圳)
HARBIN INSTITUTE OF TECHNOLOGY, SHENZHEN



Contents 内容



1 数字签名概述

2 基于RSA的数字签名

3 基于离散对数的数字签名

4 盲数字签名

2.1 数字签名算法思考的主要问题

- 选用何种公钥密码体制;
- 如何实现签名算法(私钥加密);
- 如何实现验证算法(公钥解密);
- 证明算法的正确性;
- 分析算法的效率;
- 目前针对这种签名/验证算法的安全分析。

2.2 RSA数字签名算法(初始化)

- 1. 选取两个大(满足安全要求)素数p和q，两个数长度接近且相差很大（否则安全性实质性降低！），强素数。
- 2. 计算n=p*q, φ(n)=(p-1)(q-1)
- 3. 随机选取整数e(1<e<φ(n))，满足gcd(e,φ(n)) = 1
- 4. 计算d，满足d*e ≡ 1（mod φ(n)）

注：n公开， p和q保密。

e为公钥，d为私钥。

2.2 RSA数字签名算法(签名和验证)

签名算法

1.利用一个安全的Hash函数h来产生消息摘要h(m)。

2.用签名算法计算签名$s=Sign_k(m)\equiv h(m)^d \bmod n$。

验证算法

1.首先利用一个安全的Hash函数h计算消息摘要h(m)。

2.用检验等式$h(m) \bmod n \equiv s^e \bmod n$ 是否成立，若相等签名有效，否则，签名无效。

2.2 RSA数字签名算法(正确性)

由于 $s \equiv h(m)^d \bmod n$

$d*e \equiv 1 \ (\mathrm{mod}\varphi(n))$

所以 $s^e \bmod n \equiv h(m)^{ed} \bmod n$

$\equiv h(m)^{k\varphi(n)+1} \bmod n$

$\equiv h(m) * h(m)^{k\varphi(n)} \bmod n$

$\equiv h(m) * (h(m)^{\varphi(n)})^k \bmod n$

$= h(m)$

2.3 RSA数字签名算法(举例)

- 初始化：

假设A选取p = 13，q = 11，e = 13，则有n = pq = 143，φ(n) = (p-1)(q-1) = 12×10 = 120。求解ed = 13d ≡ 1(mod 120) 得 d = 37。因此A的公钥为（n = 143, e = 13）；私钥为d = 37。

- 签名过程：

假定消息m的Hash值h(m) = 16，则计算m签名$s \equiv h(m)^d \bmod n \equiv 16^{37} \bmod 143 = 3$。

- 验证过程：

接受者B收到签名后，计算$s^e \bmod n \equiv 3^{13} \bmod 143 = 16$，$h(m) \bmod n \equiv 16 \bmod 143 = 16$，等式 $h(m) \bmod n \equiv s^e \bmod n$成立，因此，B验证此签名有效。

2.4 关于RSA数字签名方案

- 如果不使用Hash函数，则对消息m_1，m_2的签名分别为$s_1 \equiv m_1^d \bmod n$，$s_2 \equiv m_2^d \bmod n$，假设攻击者获得了这两个签名，就可以伪造消息m_1*m_2的有效签名s_1*s_2。（同态性）

  使用Hash函数可以避免这样的攻击。

- 对于大消息而言，对其Hash值的签名不仅不失数字签名特征，而且大大提高其签名和验证的效率。
- RSA签名方案还存在签名可重用的问题，即对同一消息在不同时刻签名是相同的。

  通过引入随机数来解决这个问题。

Contents 内容



1 数字签名概述

2 基于RSA的数字签名

3 基于离散对数的数字签名

4 盲数字签名

3.1 基于离散对数的签名算法

- ElGamal数字签名
- Schnorr数字签名
- **DSA数字签名**

3.2 Elgamal签名算法(初始化和签名)

- 初始化：

首先选择一个大素数p，使在Zp中求解离散对数困难。然后选择一个生成元$g \in Zp^*$，计算$y \equiv g^x \bmod p$，

则公钥：y，g，p，私钥：x。

- 签名过程：

设待签消息为m，签名者选择随机数$k \in_R Zp^*$，计算：

非0

随机

$$r \equiv g^k \bmod p$$

$$s \equiv (h(m)-xr)k^{-1} \bmod (p-1)$$

则数字签名为(r，s)，其中h为Hash函数。

3.2 Elgamal签名算法(验证)

验证过程：

签名接受者在收到消息m和签名值(r，s)后，首先计算h(m)，然后验证等式：

$$y^r r^s \equiv g^{h(m)} \bmod p$$

如等式成立，则数字签名有效；否则签名无效。

3.2 Elgamal签名算法(正确性)

因为：

$$r \equiv g^k \bmod p$$

$$s \equiv (h(m) - xr)k^{-1} \bmod (p-1)$$

所以：

$$ks \equiv h(m) - xr \bmod (p-1)$$

$$g^{ks} \equiv g^{h(m)-xr} \bmod p$$

$$g^{ks} g^{xr} \equiv g^{h(m)} \bmod p$$

$$y^r r^s \equiv g^{h(m)} \bmod p$$

$1 \equiv g^{p-1} \bmod p$

$y \equiv g^x \bmod p$

3.2 Elgamal签名算法(举例)

● 初始化：

假设A选取素数p = 19 ，Zp* 的生成元g = 2。选取私钥 x = 15，计算$y \equiv g^x \bmod p \equiv 2^{15} \bmod 19 = 12$，则A的公钥是(p = 19, g = 2, y = 12)。

● 签名过程：

设消息m的Hash值h(m) = 16，则A选取随机数k = 11，计算$r \equiv g^k \bmod p \equiv 2^{11} \bmod 19 \equiv 15$，$k^{-1} \bmod (p-1) = 5$。最后计算签名$s \equiv [h(m)-xr]k^{-1} \bmod (p-1) \equiv 5(16-15\times 15) \bmod 18 = 17$。得到A对m的签名为(15, 17)。

● 验证过程：

接受者B得到签名(15 , 17)后计算$y^r r^s \bmod p \equiv 12^{15}15^{17} \bmod 19 = 5$，$g^{h(m)} \bmod p \equiv 2^{16} \bmod 19 = 5$ 。验证等式$y^r r^s \equiv g^{h(m)} \pmod p$相等，因此B接受签名。

3.2 Elgamal签名算法(安全性)

- 不能泄露随机数k。

如果k值泄露，则容易计算：
$x \equiv [\, h(m)-sk] \, r^{-1} \bmod (p-1)$，
签名者的私钥泄露。

- 不能使用相同的k对两个不同消息进行签名。

假设k用来对两个不同的消息签名，则r相同，
即(r, s_1)是m_1的签名，
(r, s_2)是m_2的签名，
有$s_1 \equiv [\, h(m_1)-xr \,] \, k^{-1} \bmod (p-1)$ (1)
$s_2 \equiv [\, h(m_2)-xr \,] \, k^{-1} \bmod (p-1)$ (2)
那么有$(s_1-s_2) \, k \equiv [\, h(m_1)-h(m_2) \,] \bmod (p-1)$，
由于消息m_1和m_2不同，有$s_1-s_2 \neq 0 \bmod (p-1)$，
则$k \equiv [\, h(m_1)-h(m_2) \,] \, (s_1-s_2)^{-1} \bmod (p-1)$，
一旦得到k，就很容易计算签名者的私钥x。

- 签名者多次签名时所选取多个k之间无关联。

3.3 Schnorr签名算法(初始化和签名)

q是p-1的大素因子

子群的生成元

- **初始化：**

首先选择一个大素数p，使在Zp中求解离散对数困难。然后选择一个生成元$g \in Zp^*$，$g^q \equiv 1 \mod p$，最后选取随机数$1<x<q$，计算$y \equiv g^x \mod p$，则公钥为(p，q，g，y)，私钥为x。

- **签名：**

用户选择随机数k，对消息进行如下计算得签名值(e, s)。

$r \equiv g^k \mod p$

$e = h(r, m)$

$s \equiv xe+k \mod q$

3.3 Schnorr签名算法(验证)

- 验证：

  接受方在收到消息m和签名值(e,s)后，进行以下步骤：

  计算$r_1 \equiv g^s y^{-e} \bmod p$

  计算$h(r_1,m)$

  验证等式：$h(r_1,m)=e$

  如果等式成立，接受签名，否则签名无效。

3.3 Schnorr签名算法(正确性)

签名体制的正确性证明：

$$r_1 \equiv g^s\ y^{-e} \bmod p$$

$$\equiv g^s\ g^{-xe} \bmod p$$

$$\equiv g^{s-xe} \bmod p$$

$$\equiv g^{xe+k-xe} \bmod p$$

$$\equiv g^k \bmod p$$

$$\equiv r$$

$y \equiv g^x \bmod p$

$H(r_1, m) = e$

$r \equiv g^k \bmod p$； $e = h(r,m)$； $s \equiv (xe+k) \bmod q$；

3.3 Schnorr签名算法(举例)

- 初始化：

假设A选取素数p=23和q=11，其中(p-1)/q=2。选取生成元h=11∈Zp* ，并计算$g \equiv h^{(p-1)/q} \bmod p \equiv 11^2 \bmod 23=6$，则有 $g^q \bmod p \equiv [h^{(p-1)/q} \bmod p]^q \bmod p \equiv h^{(p-1)} \bmod p=1$ 。既然g≠1，那么g生成Zp* 中一个11阶循环子群。然后选取私钥x=10 ，计算$y \equiv g^x \bmod p \equiv 6^{10} \bmod 23=4$，则A的公钥是（p=23, q=11, g=6, y=4），私钥为（x=10）。

3.3 Schnorr签名算法(举例)

- 签名过程：

选取随机数$k = 9$，$1 \le k \le 10$，计算$r \equiv g^k \bmod p \equiv 6^9 \bmod 23 = 16$。设有$e = h(m||r) = 13$，计算$s \equiv (xe+k) \bmod q \equiv (10 \times 13 + 9) \bmod 11 = 7$。因此消息m的签名为（$e = 13, s = 7$）。

- 验证过程：

签名接收者B计算$r_1 \equiv g^s y^{-e} \bmod p \equiv 6^7 \times 4^{-13} \bmod 23 \equiv 16 = r$。则有 $h(m || r_1) = h(m || r) = e = 13$，因此B接受签名。

3.4 DSA数字签名

- 1994年12月美国国家标准和技术研究所(NIST，National Institute of Standard and Technology)正式颁布了数字签名标准DSS(Digital Signature Standard)，它是在ElGamal和Schorr数字签名方案的基础上设计的。在2020年5月更新的NIST SP 800-57密钥管理标准中，建议使用长度为2048比特的素数p，以及长度为224比特的素数q，并建议在2030年后使用3072比特的素数p和256比特的素数q。当p选为2048比特的素数时，ElGamal签名的长度为4096比特，而DSS通过224比特的素数q可将签名长度降低为448比特，这就减少了存储空间和传输带宽。由于DSS具有较大的兼容性和适用性，因此DSS将得到广泛的应用。数字签名标准DSS中的算法常称为DSA（Digital Signature Algorithm）。

3.4 DSA数字签名算法（初始化）

- 首先选择一个约224位比特的素数q，接着选择一个长度在约2048比特之间的素数p，使得p-1能被q整除，最后选择$g \equiv h^{(p-1)/q} \bmod p$，其中h是整数，是$Z_p^*$的生成元，满足$1 < h < p-1$，且$g > 1$。
- 用户A选择1到q之间的随机数x作为用户的私钥，计算$y \equiv g^x \bmod p$，用户的公钥为（p，q，g，y）。

3.4 DSA数字签名算法（签名）

- 用户选择随机数k，对消息m，计算两个分量r和s产生签名值(r，s)：

$$r \equiv (g^k \bmod p) \bmod q;$$

$$s \equiv [H(m)+xr]k^{-1} \bmod q;$$

- 其中H为Hash函数，DSS标准中规定了Hash算法为SHA224或SHA512/224算法。

3.4 DSA数字签名算法（验证）

接受方在收到消息m和签名值(r,s)后，进行以下计算步骤：

$$w \equiv s^{-1} \bmod q ;$$

$$u_1 \equiv [h(m)w] \bmod q ;$$

$$u_2 \equiv rw \bmod q ;$$

$$v = \left(g^{u_1} y^{u_2} \bmod p \right) \bmod q$$

将v和r进行比较，若v = r，则签名有效；否则，签名无效。

3.4 DSA数字签名算法（正确性）

因为： $r \equiv (g^k \bmod p) \bmod q$;

$s \equiv [h(m)+xr]k^{-1} \bmod q$;

$w \equiv s^{-1} \bmod q$;

$u_1 \equiv [h(m)w] \bmod q$;

$u_2 \equiv rw \bmod q$;

$$v = \left(g^{u_1} y^{u_2} \bmod p \right) \bmod q$$

所以： $v = \left(g^{u_1} y^{u_2} \bmod p \right) \bmod q \equiv [(g^{h(m)w} y^{rw}) \bmod p] \bmod q$

$\equiv [(g^{h(m)w} g^{xrw}) \bmod p] \bmod q \equiv [g^{(h(m)+xr)w} \bmod p] \bmod q \equiv$

$(g^{skw} \bmod p) \bmod q \equiv (g^k \bmod p) \bmod q = r$

3.4 DSA数字签名算法（举例）

初始化：

假设A选取素数$p = 23$，$q = 11$，其中$(p-1)/q = 2$。选择随机数$h = 11 \in_R Zp^*$，计算$g \equiv h^{(p-1)/q} \bmod p \equiv 11^2 \bmod 23 = 6$。既然$g \neq 1$，那么g生成$Zp^*$中唯一的q阶循环子群。接着选择随机数$x = 10$满足$1 \leq x \leq q-1$，并计算$y \equiv g^x \bmod p \equiv 6^{10} \bmod 23 = 4$。则公钥为（$p = 23, q = 11, g = 6, y = 4$），私钥为（$x = 10$）。

签名过程：

选取随机数$k = 9$，计算$r \equiv (g^k \bmod p) \bmod q \equiv (6^9 \bmod 23) \bmod 11 = 5$。然后计算$k^{-1} \bmod q = 5$。假设$h(m) = 13$，计算$s \equiv [h(m)+xr]\, k^{-1} \bmod q \equiv 5 \times (13+10 \times 5) \bmod 11 = 7$。因此消息m的签名为（$r = 5, s = 7$）。

验证过程：

签名接收者B计算$w \equiv s^{-1} \bmod q = 8$，$u_1 \equiv [h(m)w] \bmod q \equiv 13 \times 8 \bmod 11 = 5$，$u_2 \equiv rw \bmod q \equiv 5 \times 8 \bmod 11 = 7$。然后计算 $v = \left(g^{u_1} y^{u_2} \bmod p\right) \bmod q \equiv (6^5 4^7 \bmod 23) \bmod 11 \equiv 5 = r$。所以B接受签名。

3.5 基于离散对数签名算法的性能对比

签名体制	签名	验证	签名值长度
ElGamal	$T_E+T_H+2T_M+T_I$	$3T_E+T_H+T_M$	$\|p\|+\|p-1\|$
Schnorr	$T_E+T_H+T_M$	$2T_E+T_H+T_M+T_I$	$\|q\|+\|h(m)\|$
DSA	$T_E+T_H+2T_M+T_I$	$2T_E+T_H+3T_M+T_I$	$2\|q\|$

- T_E：幂运算的计算量
- T_H：哈希计算的计算量
- T_M：乘积运算的计算量
- T_I：求逆运算的计算量
- 其余的运算与上述四种运算相比可忽略不计

Contents 内容



1 数字签名概述

2 基于RSA的数字签名

3 基于离散对数的数字签名

4 盲数字签名

4.1 特殊数字签名的简介

- 在实际应用当中，一些特殊的场合往往对数字签名有特殊的需求，因此，需要在基本数字签名技术的基础上进行扩展，以满足这些特殊的需求。
- 为保护信息拥有者的隐私，产生了盲签名；为了实现签名权的安全传递，产生代理签名；为了实现多人对同一个消息的签名，产生了多重签名等等。
- 这些实际应用的特殊需求，使得各种各样的特殊数字签名的研究一直是数字签名研究领域非常活跃的部分并产生很多分支。

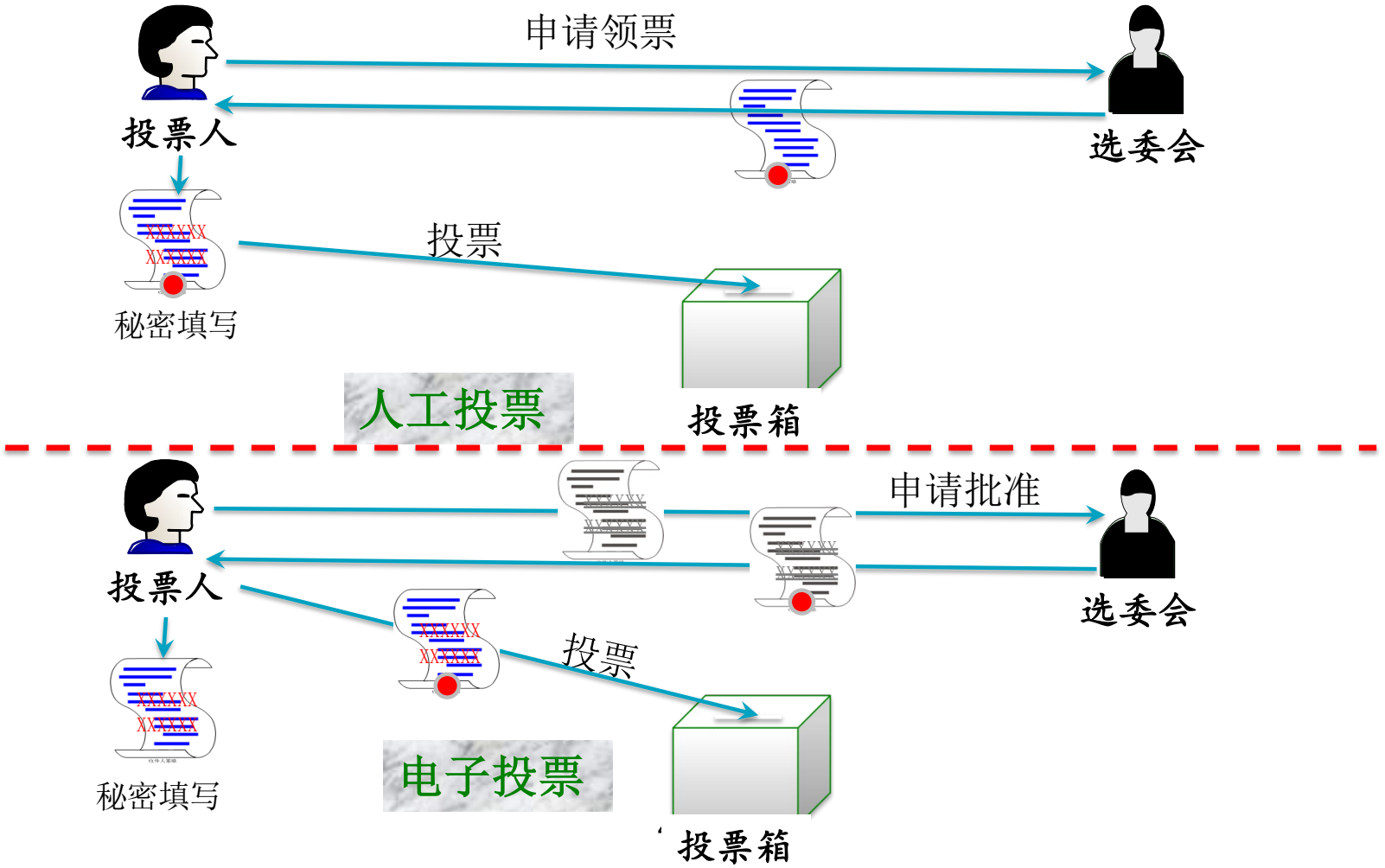
42

4.2 盲签名-简介

- 盲签名是D.Chaum于1982年首次提出的一种具有特殊性质的数字签名
- 这种签名要求签名者能够在不知道被签名文件内容的情况下对消息进行签名。另外，即使签名者在以后看到了被签名的消息及其签名，签名者也不能判断出这个签名是他何时为谁生成的
- 直观上讲，这种签名的生成过程就像签名者闭着眼睛对消息签名一样，所以形象地称为“盲”数字签名。
- 盲签名在电子货币、电子投票、电子拍卖等应用中发挥了重要作用，因为它解决了人们关注的个人隐私问题。

43

4.1 盲签名-投票场景



4.2 盲签名-特点

- **匿名性**

签名者对这个消息签了名，如果把消息签名给签名者看，签名者也确信这个签名是有效的，**这个消息签名具有数字签名的所有性质**，但签名者无法知道所签消息的具体内容。

事先不知

- **不可追踪性**

签名者不能把签署消息的行为与签署的消息相关联，即使他记下了他所作的每一个盲签名，他也不能把某个消息的盲签名与某个消息的内容相关联。

事后不知

45

4.1 盲签名-防止请求者滥用

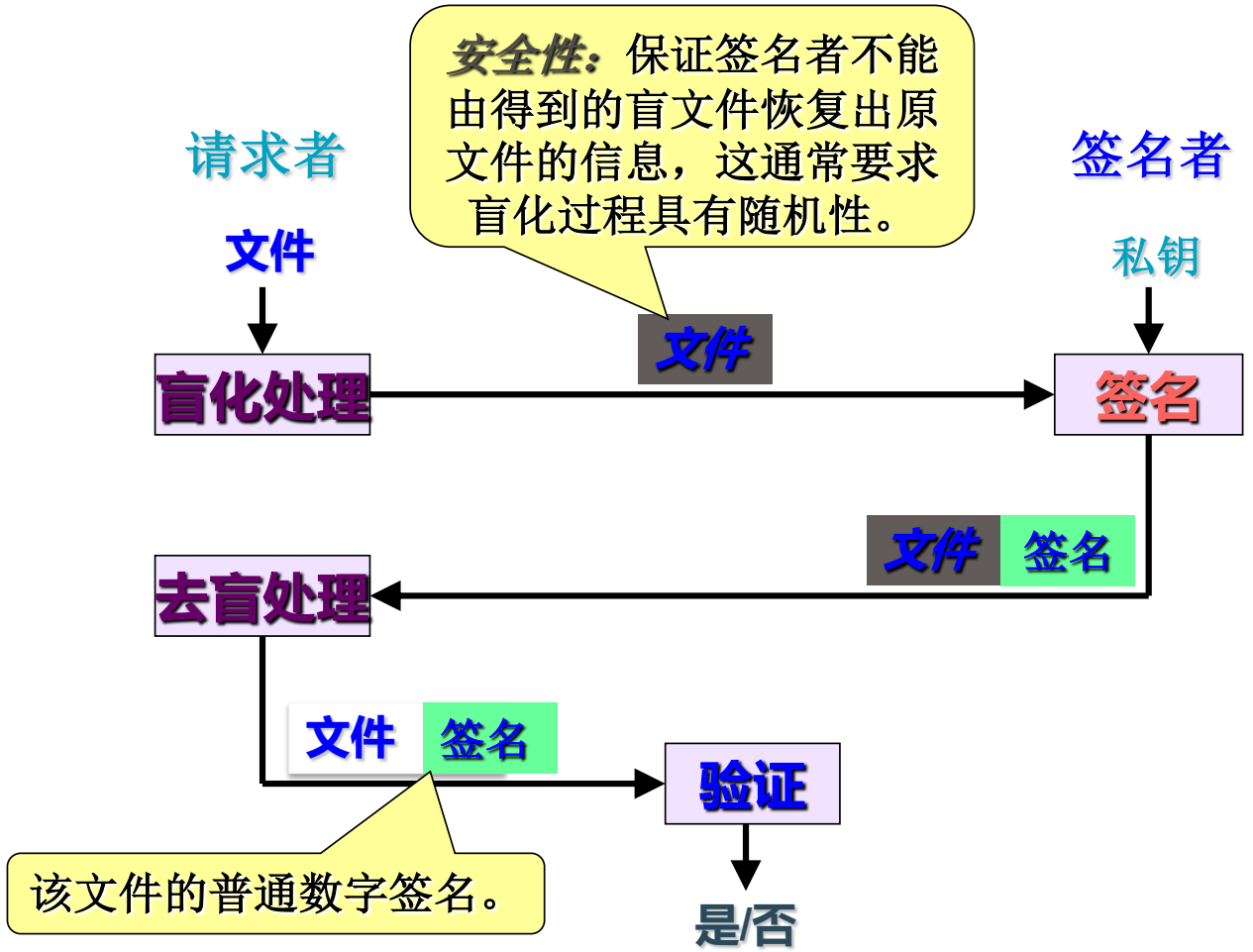
- 通常情况下，请求者只有满足一定条件，签名者才能完成签名，而且通常也规定签名的范围，也就是说，通常要有明确约束条件。

  譬如：

  - ✓ 在电子投票，投票者只有具有投票资格且之前没有申请过投票，选委会才能对投票者提供的“选票”进行签字；
  - ✓ 在电子货币，消费者的账户金额只有多余其申请消费金额，银行就才对消费者的这笔“消费”进行签字。

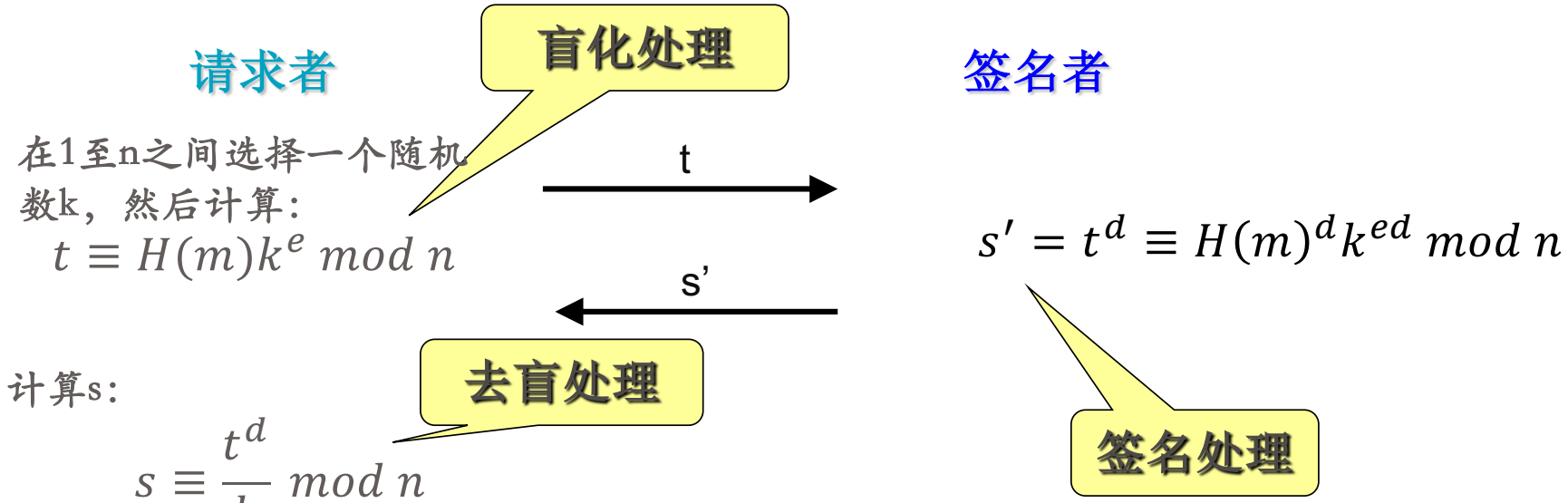
46

4.2 盲签名-示意过程



47

4.2 盲签名-盲RSA签名算法



计算s：

$$s \equiv \frac{t^d}{k} \bmod n$$

结果：

$$s \equiv H(m)^d \bmod n$$

容易证明：

$$s \equiv td/k \bmod n \equiv (H(m)k^e)^d/k \bmod n \equiv H(m)^d \bmod n$$

总结：消息认证码和数字签名的对比

	消息认证码	数字签名
发送者	用对称密钥计算MAC	用私钥生成签名
接受者	用对称密钥计算MAC	用公钥验证签名
密钥分发问题	存在	不存在，但公钥需要另外认证
效率	高	低
完整性	支持	支持
认证性	支持(仅限通信双方)	支持(可适用于信任第三方)
不可否认性	不支持	支持

课后



1、请描述数字签名满足不可否认性的原理和依据。

2、列举基于RSA算法数字签名的特点和安全缺陷。

3、验证一种基于离散对数的数字签名的正确性。



谢 谢